



Memoria Técnica del Proyecto

Alumno/a	Manuel Rossi López
Ciclo	Desarrollo de Aplicaciones Web (DAW)
Curso	1º DAW
Profesor/a	Esteban Castañer Pardo, Jose Luiz López de Pablo Moya, Pedro Campos Cuevas
Fecha de entrega	31/5/2026

Módulos implicados

Programación · Bases de Datos · Lenguajes de Marcas · Entornos de Desarrollo · Sistemas Informáticos

2. Introducción

LoveCode es una aplicación web desarrollada como proyecto integrador para el ciclo formativo de Desarrollo de Aplicaciones Web de 1º DAW.

La aplicación está inspirada en las plataformas de conexión social, pero orientada exclusivamente a desarrolladores informáticos. Su objetivo es permitir que los usuarios puedan registrarse con sus datos personales, visualizar perfiles de otros programadores y establecer conexiones profesionales mediante un sistema de likes y matches.

El proyecto pretende integrar de forma práctica los conocimientos adquiridos durante el curso, especialmente en los módulos de Programación, Bases de Datos y Lenguajes de Marcas.

Funcionalidades principales

- Registro de nuevos usuarios con datos personales.
- Inicio de sesión con autenticación contra la base de datos.
- Visualización de perfiles de otros desarrolladores.
- Gestión de tecnologías asociadas a cada usuario.
- Sistema de likes entre usuarios.
- Generación automática de matches cuando dos usuarios se dan like mutuamente.
- Generación de archivos XML cuando se produce un match.

3. Descripción General del Sistema

La aplicación está formada por tres componentes principales que trabajan en conjunto para ofrecer toda la funcionalidad del sistema.

Arquitectura general

Capa	Tecnología	Responsabilidad
Base de datos	MySQL / MariaDB	Persistencia de datos
Backend	Java 17 + JDBC	Lógica de negocio y servidor HTTP
Frontend	HTML5 + CSS3 + JavaScript	Interfaz de usuario

Base de datos

La base de datos está desarrollada en MySQL y almacena toda la información relativa a usuarios, tecnologías, likes y matches. También incorpora triggers y procedimientos almacenados para automatizar operaciones.

Backend Java

El backend está desarrollado íntegramente en Java 17 usando JDBC para la conexión con la base de datos. Incluye un servidor HTTP construido sobre `com.sun.net.httpserver.HttpServer`, que forma parte del propio JDK de Java y no requiere ninguna dependencia externa.

Frontend web

El frontend está desarrollado con HTML5, CSS3 y JavaScript puro, sin frameworks adicionales. Consta de tres páginas principales: login, registro y visualización de perfiles.

Comunicación entre capas

La comunicación entre el frontend y el backend se realiza mediante peticiones HTTP usando la función `fetch()` de JavaScript. El servidor Java expone una API bajo la ruta `/api/` y también sirve directamente los archivos estáticos del frontend, por lo que todo el sistema funciona desde un único puerto (8080).

4. Diseño de la Base de Datos

Tablas principales

Tabla: usuarios

Almacena la información personal de cada usuario registrado en la plataforma.

Campo	Tipo	Restricción	Descripción
id	INT	PK, AUTO_INCREMENT	Identificador único
nombre	VARCHAR(100)	NOT NULL	Nombre del usuario
apellido1	VARCHAR(100)	NOT NULL	Primer apellido
apellido	VARCHAR(100)	NULL	Segundo apellido
email	VARCHAR(150)	UNIQUE, NOT NULL	Correo electrónico
contrasena	VARCHAR(255)	NOT NULL	Contraseña del usuario
fecha_nacimiento	DATE	NOT NULL	Fecha de nacimiento

Tabla: tecnologias

Contiene el catálogo de tecnologías disponibles en la plataforma.

Campo	Tipo	Restricción	Descripción
id	INT	PK, AUTO_INCREMENT	Identificador único
nombre_tecnologia	VARCHAR(100)	NOT NULL	Nombre de la tecnología
tipo	VARCHAR(50)	NULL	Categoría de la tecnología

Tabla: likes

Registra los likes enviados entre usuarios.

Campo	Tipo	Restricción	Descripción
id	INT	PK, AUTO_INCREMENT	Identificador único
id_usuario_da	INT	FK → usuarios.id	Usuario que da el like
id_usuario_recibe	INT	FK → usuarios.id	Usuario que recibe el like

Tabla: matches

Almacena los matches generados automáticamente cuando dos usuarios se dan like

mutuamente.

Campo	Tipo	Restricción	Descripción
id	INT	PK, AUTO_INCREMENT	Identificador único
id_usuario1	INT	FK → usuarios.id	Primer usuario del match
id_usuario2	INT	FK → usuarios.id	Segundo usuario del match

Trigger de generación de matches

Se implementó el trigger `crear_match`, que se ejecuta automáticamente después de cada `INSERT` en la tabla `likes`. Su lógica comprueba si existe un like recíproco entre los dos usuarios involucrados. Si ambos se han dado like mutuamente y todavía no existe un registro de match entre ellos, el trigger inserta automáticamente un nuevo registro en la tabla `matches`.

Esto garantiza que los matches se generen de forma transparente y sin necesidad de lógica adicional en la aplicación.

Procedimientos almacenados

- `conteo_matches`: cuenta el número total de matches de un usuario dado su `id`.
- `cargar_usuario`: devuelve la información completa de un usuario junto con sus tecnologías asociadas.
- `borrar_usuario`: elimina un usuario y todos sus datos relacionados de la base de datos.

5. Desarrollo del Backend

Estructura de clases

Clase	Responsabilidad
ConexionBD	Gestiona la conexión JDBC con MySQL
Usuario	Modelo de datos del usuario
Tecnologia	Modelo de datos de tecnología
UsuarioService	Login, registro, obtención de perfiles
TecnologiaService	Consulta de tecnologías
LikeService	Registro de likes y detección de match
MatchService	Verificación y conteo de matches
XmlService	Generación de archivos XML por match
Servidor	Servidor HTTP con los endpoints de la API
Main	Punto de entrada: arranca el servidor HTTP

Conexión con la base de datos

La conexión se realiza en la clase ConexionBD usando JDBC con el driver MySQL Connector/J incluido en la carpeta lib/. El driver se carga explícitamente mediante `Class.forName` para garantizar su disponibilidad cuando el proyecto se ejecuta con Maven.

```
Class.forName("com.mysql.cj.jdbc.Driver");
conexion = DriverManager.getConnection(URL, USER, PASSWORD);
```

Servidor HTTP

El servidor HTTP se construye con `com.sun.net.httpserver.HttpServer`, disponible en el JDK sin necesidad de dependencias externas. Escucha en el puerto 8080 y registra los siguientes contextos:

- `/api/login` — Autenticación de usuarios (POST).
- `/api/registro` — Creación de nuevos usuarios (POST).
- `/api/perfiles` — Listado de perfiles excluyendo al usuario actual (GET).
- `/api/like` — Registro de un like y detección de match (POST).
- `/` — Sirve los archivos estáticos del frontend (GET).

Todas las respuestas de la API incluyen las cabeceras CORS necesarias para que el navegador permita las peticiones desde cualquier origen.

Gestión del login

El endpoint `/api/login` recibe un JSON con email y contraseña, delega la verificación en `UsuarioService.login()` y devuelve el id y nombre del usuario si las credenciales son correctas, o un código HTTP 401 en caso contrario.

Gestión del registro

El endpoint `/api/registro` recibe los datos del formulario, los valida, convierte la fecha al tipo `java.sql.Date` y llama a `UsuarioService.registrarUsuario()` para insertar el nuevo usuario en la base de datos.

Sistema de likes

Cuando se recibe una petición POST a `/api/like`, el servidor llama a `LikeService.darLike()`, que inserta el like en la tabla `likes`. Después comprueba mediante `MatchService.existeMatch()` si existe un match recíproco y devuelve el resultado al frontend.

Generación de XML

Cuando `LikeService` detecta un match, llama a `XmlService.generarXML()`, que crea un archivo XML en la carpeta `matches_xml/` con el formato `match_idUsuario1_idUsuario2.xml`, registrando los identificadores de ambos usuarios involucrados en el match.

6. Desarrollo del Frontend

Estructura de archivos

Archivo	Función
login.html	Página de inicio de sesión
registro.html	Página de registro de nuevos usuarios
perfiles.html	Página de visualización de perfiles con likes
css/style.css	Hoja de estilos única para todas las páginas
js/App.js	Lógica JavaScript: validación, fetch y renderizado
tests.html	Pruebas unitarias en JavaScript puro

Página de login

Formulario con campos email y contraseña. Al enviar, la función login() de App.js valida los datos y realiza una petición POST a /api/login. Si la respuesta es correcta, guarda el id y nombre del usuario en sessionStorage y redirige a perfiles.html.

Página de registro

Formulario con los campos exactos de la tabla usuarios en la base de datos: nombre, apellido1, apellido, email, contraseña y fecha_nacimiento. La función registrarUsuario() valida localmente antes de enviar la petición al servidor.

Página de perfiles

Muestra una cuadrícula de tarjetas con los perfiles de otros usuarios. Cada tarjeta incluye las iniciales del usuario, su nombre completo, su edad calculada dinámicamente y dos botones de acción: Like y Pasar.

Al pulsar Like, se envía una petición POST a /api/like. Si el servidor devuelve que hay match, se muestra una notificación emergente en la esquina inferior derecha de la pantalla.

Comunicación con el backend

Toda la comunicación se realiza mediante la función fetch() de JavaScript. La URL base de la API está definida en una constante al inicio de App.js, facilitando el mantenimiento:

```
const API_URL = 'http://localhost:8080/api';
```

El frontend envía y recibe datos en formato JSON. Los errores de conexión se capturan con .catch() y se muestran al usuario mediante mensajes en pantalla.

Diseño visual

El diseño utiliza un tema oscuro con los colores corporativos del proyecto: rojo oscuro (#C0392B) como color principal y fondos en tonos de gris oscuro. Se usa la fuente Space Mono para títulos y DM Sans para el texto general, ambas de Google Fonts. El diseño es completamente responsivo mediante CSS Grid y media queries.

7. Automatización del Sistema

Script de backup

Se desarrolló un script que realiza una copia de seguridad automática de la base de datos. El script utiliza el comando mysqldump para exportar tanto la estructura como los datos de todas las tablas, guardando el resultado en un archivo .sql con marca de fecha y hora para facilitar su identificación.

Script de arranque

Se implementó un script de arranque que automatiza la puesta en marcha del sistema. Permite iniciar el backend Java con un único comando sin necesidad de recordar los parámetros de Maven, y muestra en la terminal la URL de acceso al sistema.

Para arrancar el proyecto manualmente se ejecutan los siguientes comandos desde la carpeta backend/:

```
mvn compile
mvn exec:java "-Dexec.mainClass=com.lovecode.Main"
```

Una vez arrancado, la aplicación es accesible directamente en:

```
http://localhost:8080/login.html
```

8. Funcionamiento del Proyecto

Registro de usuario

El usuario accede a la página de registro en /registro.html, completa el formulario con su nombre, apellidos, email, contraseña y fecha de nacimiento, y pulsa el botón Crear cuenta. El frontend valida los datos localmente y los envía al servidor. Si el registro es correcto, el sistema redirige automáticamente al login.

Inicio de sesión

El usuario introduce su email y contraseña en /login.html. El sistema verifica las credenciales contra la base de datos. Si son correctas, guarda el identificador del usuario en sessionStorage y redirige a la página de perfiles.

Visualización de perfiles

En /perfiles.html el sistema carga automáticamente todos los usuarios registrados excepto el usuario actual. Cada perfil se muestra en una tarjeta con el avatar generado a partir de las iniciales, el nombre completo y la edad calculada a partir de la fecha de nacimiento.

Sistema de likes

El usuario puede pulsar el botón Like en cualquier tarjeta de perfil. La acción registra el like en la base de datos y la tarjeta desaparece de la pantalla. Si pulsa Pasar, la tarjeta simplemente se oculta sin registrar nada en el servidor.

Generación de match

Cuando dos usuarios se dan like mutuamente, el trigger de la base de datos crea automáticamente un registro en la tabla matches y el sistema genera un archivo XML en la carpeta matches_xml/. En la interfaz, el usuario que da el segundo like ve aparecer una notificación emergente informándole del match.

9. Problemas Encontrados y Soluciones

Driver MySQL no encontrado al ejecutar con Maven

Al arrancar el servidor con `mvn exec:java`, Java lanzaba el error `No suitable driver found for jdbc:mysql`. El problema era que el classloader de `exec:java` no registraba automáticamente el driver JDBC.

Solución: Se añadió la línea `Class.forName("com.mysql.cj.jdbc.Driver")` al inicio del método `conectar()` en `ConexionBD.java` para forzar la carga explícita del driver.

Conflicto entre rutas de API y archivos estáticos

Al intentar servir el frontend desde el mismo servidor Java, la ruta `/login` capturaba las peticiones a `/login.html`, devolviendo un JSON de error en lugar del HTML de la página.

Solución: Se movió toda la API bajo el prefijo `/api/`, de forma que `/api/login` gestiona la autenticación y `/login.html` sirve el archivo HTML sin conflictos.

Error ERR_EMPTY_RESPONSE en el navegador

El navegador mostraba `ERR_EMPTY_RESPONSE` al intentar acceder al servidor. El problema era que el servidor Java se caía al intentar procesar la petición debido a una excepción no capturada en el handler.

Solución: Se añadieron bloques `try-catch` completos en todos los handlers del servidor para garantizar que siempre se envíe una respuesta HTTP, incluso en caso de error.

Frontend abierto desde `file://` bloqueaba `fetch()`

Al abrir los archivos HTML directamente con doble clic, el navegador los cargaba con el protocolo `file://`, lo que bloqueaba las peticiones `fetch()` al servidor por restricciones de seguridad del navegador.

Solución: Al servir el frontend directamente desde el servidor Java en el puerto 8080, todos los archivos se cargan bajo `http://localhost:8080`, eliminando el problema de CORS y de protocolo `file://`.

10. Conclusiones

La realización de LoveCode ha permitido aplicar de forma práctica y conjunta los conocimientos adquiridos durante el curso en los diferentes módulos del ciclo formativo. El resultado es una aplicación web funcional con backend, base de datos y frontend integrados y comunicados entre sí.

Se han utilizado tecnologías reales empleadas en entornos profesionales: Java con JDBC para el backend, MySQL con triggers y procedimientos almacenados para la persistencia, y HTML, CSS y JavaScript puro para la interfaz de usuario.

Las partes más complejas del desarrollo han sido la integración entre el frontend y el backend mediante HTTP, la resolución de los problemas de CORS y classloading de JDBC, y la automatización de los matches mediante el trigger de MySQL.

Aprendizajes principales

- Implementación de un servidor HTTP en Java puro sin frameworks externos.
- Comunicación cliente-servidor mediante fetch() y JSON.
- Gestión de triggers y procedimientos almacenados en MySQL.
- Organización de un proyecto Java con Maven.
- Diseño de interfaces web responsivas sin frameworks frontend.

Posibles mejoras futuras

- Encriptación de contraseñas con bcrypt.
- Sistema de mensajería privada entre usuarios con match.
- Fotografías de perfil subidas por el usuario.
- Filtrado de perfiles por tecnologías.
- API REST completa con autenticación mediante tokens JWT.

11. Anexos

Estructura del repositorio

```

LOVECODE_PROYECTO/
├── lovecode/
│   ├── backend/
│   │   ├── lib/mysql-connector-j.jar
│   │   ├── matches_xml/
│   │   ├── src/main/java/com/lovecode/
│   │   │   ├── database/ConexionBD.java
│   │   │   ├── models/Usuario.java
│   │   │   ├── models/Tecnologia.java
│   │   │   ├── servicies/UsuarioService.java
│   │   │   ├── servicies/LikeService.java
│   │   │   ├── servicies/MatchService.java
│   │   │   ├── servicies/XmlService.java
│   │   │   ├── Servidor.java
│   │   │   └── Main.java
│   │   └── pom.xml
│   └── frontend/
│       ├── css/style.css
│       ├── js/App.js
│       ├── login.html
│       ├── registro.html
│       ├── perfiles.html
│       └── tests.html
└── database/

```

Comandos de ejecución

Comando	Descripción
mvn compile	Compila el proyecto Java
mvn exec:java -Dexec.mainClass=...	Arranca el servidor HTTP
http://localhost:8080/login.html	URL de acceso a la aplicación

Tecnologías utilizadas

Tecnología	Versión	Uso
Java	17	Backend y servidor HTTP
Maven	3.x	Gestión del proyecto
MySQL / MariaDB	8.x	Base de datos
JDBC	—	Conexión Java-MySQL
HTML5	—	Estructura del frontend
CSS3	—	Estilos y diseño responsivo
JavaScript	ES6+	Lógica del frontend y fetch()
GitHub	—	Control de versiones

